Gerenciador de Buffer

Implementação em C/C++ de Políticas LRU, FIFO, CLOCK e MRU para Sistemas de Memória

Visão Geral do Projeto

- **Simulação de Memória:** O sistema simula um gerenciador de buffer onde as páginas são lidas sob demanda.
- **Armazenamento Secundário:** Um arquivo de texto CSV (.txt) atua como o disco rígido contendo os dados originais.
- **Métricas de Desempenho:** O gerenciador rastreia variáveis vitais como **cache hit** e **cache miss** para avaliar a eficiência.
- **Substituição Dinâmica:** Quando a memória fica cheia, algoritmos determinam qual página será despejada (evicted) baseados no histórico de acesso.

Estrutura da Entrada (Page)

Page ID

Identificador único da página do Struct (`page#`). Funciona como a chave primária para buscas rápidas em memória.

Content

O conteúdo real extraído do arquivo de texto. Representa os dados em cache prontos para serem servidos à CPU.

Dirty Bit

Variável de atualização booleana. Se `TRUE` , indica que a página foi modificada e precisa ser gravada no disco antes do descarte.

Políticas de Substituição

Baseadas em Frequência (LRU / MRU)

Least Recently Used (LRU): Descarta a página que não foi acessada há mais tempo. Ideal para padrões com forte localidade temporal.
Most Recently Used (MRU): Descarta a página acessada mais recentemente. Utilidade em cenários específicos como loops de banco de dados onde o cache mais antigo é preferível.

Baseadas em Inserção (FIFO / CLOCK)

First-In, First-Out (FIFO): Descarta simplesmente a página mais antiga no buffer, sem olhar os acessos.
CLOCK (Second Chance): Usa um ponteiro circular e um `bit de referência`. Garante uma "segunda chance" a páginas recentemente acessadas, aproximando-se do LRU com menor custo de CPU.

Operação: Fetch(key)

A função `Fetch(int key)` é o coração do sistema, responsável por buscar os dados solicitados.

Se a chave estiver no buffer (Hit): Retorna o conteúdo instantaneamente e atualiza a ordem das listas (para LRU/MRU) ou os bits de referência `Dirty` (CLOCK).

Se não estiver (Miss): Busca a linha no arquivo de texto e a insere. Um valor aleatório é gerado. Caso o buffer tenha atingido sua capacidade máxima, invoca a função `Evict()` antes da inserção.

Evicção e Monitoramento

Evict() Remove uma página obedecendo estritamente a política escolhida na execução. Se a página removida estiver com o bit de atualização ativo, imprime "W" no console, simulando o *write-back* no disco.

DisplayCache() Fornece uma visualização formatada do estado atual do buffer, exibindo a Chave (page#), Valor (texto truncado para legibilidade) e o estado do Dirty Bit de cada slot na memória.

DisplayStats() Gera o relatório de eficiência do gerenciador. Exibe os totais e percentuais absolutos de (leituras rápidas) e (penalidades de disco).

`Cache Hit` `Cache Miss`